AI经纪人部署文档（粉丝版）

AI 经纪人 · 内容流水线部署文档（粉丝版）

> **这份文档是什么**：一套「Obsidian 内容流水线」的完整安装包。你的灵感记进手机后，AI 经纪人每天自动帮你入库、写大纲、倒排期、同步苹果日历，你只管照着日历创作。
>
> **怎么用**：把整份文档发给你的 AI agent（WorkBuddy、Codex 或任何能操作你电脑的 agent），对它说「照着这份文档帮我部署」，然后回答它问你的几个问题。剩下的它全包。
>
> **前提**：Mac 电脑 + iPhone（同一 Apple ID 登录 iCloud）+ 已安装 Obsidian。定时任务只在 Mac 开机时运行。
>
> 版本：v1.0（2026-09）

第一部分 · 系统一图流（人看的）

```
📱 手机                                  🖥️ Mac（全自动，无需操作）
──────                                   ──────────────────────────
随手录入灵感 ── iCloud 同步 ──→ 每天 9:00   灵感入库（生成选题卡）
   （30 秒）                              每天 9:05   倒排期（脚本→拍/素材→剪→发）
                                          每天 9:10   同步苹果日历（手机可见）
看板/日历/每日小记 ←──────────── 全部落回 Obsidian，手机随时查看
晚上勾掉当天任务 ──→ 21:00 同步完成状态（看板移「已完成」+ 日历打勾）
```

核心原则：Obsidian 是唯一数据源头。 所有时间改动只在 Obsidian 里做，苹果日历是只读看板——在手机日历上手动改的会被次日 9:10 的同步覆盖。

部署完你每天的生活：

时段	动作	耗时
随时（碎片）	打开 Obsidian → 星标文件「随手录入」→ 顶部空行写一条。链接/口述/截图描述都行，不用整理格式	30 秒/条
早上（可选）	看手机日历或「任务总览」，知道今天该干嘛	30 秒
晚上（收工）	打开当天「每日小记」→ 做完的任务勾 `[x]` → 21:00 自动同步	1 分钟
每周日	「任务总览」→ Timeline 未来 14 天 → 调发布日期	5 分钟

第二部分 · 部署指令（AI Agent 看的）

1. 角色与目标

你是部署 agent。你的任务：在用户的 Mac 上部署一套「Obsidian 内容流水线」系统，包括——创建 vault 及目录结构、写入全部配置文件与指令文档、安装 Obsidian 插件、部署 4 个定时任务。按本文档步骤顺序执行，遇到与用户环境相关的信息先问再动。

2. 前置检查（不满足先报告，不要硬装）

1. **macOS**（依赖 launchd 定时任务和 osascript 日历脚本）
2. **iCloud Drive 已登录**（vault 必须放在 iCloud 路径下才能手机电脑同步）
3. **Obsidian 已安装**（https://obsidian.md）
4. **AI agent CLI 可用**：检测你自己的可执行文件绝对路径（如 WorkBuddy 内置的 `codebuddy`，或用户安装的 `codex` 等），记下来，代入下文所有 `{{AGENT_CLI}}`。要求支持非交互模式执行带 prompt 的命令和 `--add-dir` 参数；如参数名不同，按你的 CLI 语法等价改写脚本
5. **Python 3** 可用（日历同步脚本需要）

3. 开始前问用户这 4 个问题

把答案记录下来，代入下文所有占位符：

1. **知识库名字**叫什么？（建议：`我的知识库`；记为 `{{VAULT_NAME}}`）
2. **vault 放在哪**？建议在 iCloud 云盘下新建文件夹（默认路径：`~/Library/Mobile Documents/com~apple~CloudDocs/{{VAULT_NAME}}`；记为 `{{VAULT_PATH}}`）
3. **同步到哪个苹果日历**？必须选 **iCloud 账户下**的日历（本地日历不进 iCloud，手机看不到）。建议为用户在 iCloud 下新建一个名为「内容排期」的日历（记为 `{{CAL_NAME}}`）
4. **默认发布时间**？（默认 18:00，记为 `{{PUBLISH_TIME}}`）

4. 部署步骤

Step 1 · 创建 vault 与目录结构

在 `{{VAULT_PATH}}` 创建 vault（用 Obsidian「打开文件夹作为仓库」或直接建目录），目录结构如下：

```
{{VAULT_NAME}}/（vault 根目录）
├── 选题灵感库/
│   ├── 0-随手录入.md          ← 手机端唯一灵感入口
│   └── _早间入库说明.md        ← 9:00 入库任务的 agent 指令
```

```
├── 时间管理/
│   └── _早间排期指令.md          ← 9:05 倒排期 + 21:00 状态同步的 agent 指令
│   └── 3-内容流程看板.md         ← Kanban 看板
├── 环节档案/                     ← 写脚本/拍摄/剪辑笔记（自动生成）
├── 每日小记/                     ← 每天任务清单，晚上在这里勾 ✅
├── 内容日历/
│   └── 流程日历/                 ← 日历事件笔记（自动生成）
├── 模板/
│   ├── 选题灵感卡.md
│   └── 每日小记.md
└── 附件/                         ← 分镜表粘贴图片自动存这里
```

Step 2 · 写入核心文件（共 7 份，全文见下）

文件 1 · `选题灵感库/0-随手录入.md`

```
# 📱 灵感
- [ ]
```

（就这么多。用户每条灵感写成一行 `- [ ] 内容`。agent 处理过的行会自动打勾并在下方加注释，原文永不删除。）

文件 2 · `选题灵感库/_早间入库说明.md`

```
# 灵感自动入库（功能一配置说明）

> 目标：手机 Obsidian `选题灵感库/0-随手录入.md` 攒的灵感/链接，由每天 9:00 的定时任务自动解析成选题卡，写入本目录。
> 状态流转：💡 灵感 → 🟡 可评估 → 🟠 已立项 → 🟢 已排期 → 🎬 脚本草稿 → 📽️ 拍摄中 → ✂️ 剪辑中 → ✅ 已发布

## 一、白天怎么记（手机上，尽量省事）

- 手机打开 Obsidian → 进入 `选题灵感库/0-随手录入.md`（在 Obsidian 启动器里把它标为星标，一键直达）
- 刷到想拍的内容，**直接把链接粘进来**，可以补一句"为什么想拍"；或者直接口述一段（语音转文字粘过来）；一条一行，不用整理格式
- 示例：
  ```
  https://v.douyin.com/xxxx 这条"日历复盘"角度想拍
  口述:看到一个博主讲ai字幕,我想做一个对比测评
  口述:旅行行程做成小程序,教程形式分享,想 9月20日 发
  ```
- **想直接排期就带上发布日期**：在灵感行里写上发布日期（`📅 2026-09-20` 或 `9月20日`
```

或 `09/20` 都行），agent 入库时会跳过选题池，直接倒排档期 + 归档到「📦 已排期」。不带日期就走常规流程（进选题池，等手动写日期）。
- iCloud 自动同步到 Mac，排期任务读得到。

二、每天 9:00 定时入库（launchd 触发）

- 入口文件：`选题灵感库/0-随手录入.md`
- **扫描范围**：整个文件中**所有 `- [ ]` 未勾选的行**（`- [x]` 已勾选的视为"已处理"，跳过）。空行（只有 `- [ ]` 符号，无文字内容）跳过不处理。
- **解析流程**：把每条 `- [ ]` 行的内容解析成一张标准选题卡，写入 `选题灵感库/` 目录：
  - 文件名：`YYYY-MM-DD 关键词.md`（用当天日期）
  - 按 `模板/选题灵感卡.md` 的字段生成：状态固定 `💡 灵感`；分类从模板给的 5 类里选；来源 = 原链接（口述无链接就写"随手录入口述转写"）；「一句话灵感」= 用户补的那句，没补就根据内容概括；「想到的钩子/切入角度」给出建议值；评分先填 0
  - **每张卡必须带上原始口述**：把 `0-随手录入.md` 里该条灵感**对应的原文逐字**复制进卡片的「## 🎙️ 原始口述」区块。链接条目就贴那一行的链接+备注；纯口述条目就整段贴转写原文。不改写、不缩写、不修口误——这一段是留给用户双击卡片时看"当初自己原话"的。
  - 同一链接如果库里已有卡片，跳过不重复建；口述条目若内容和已有卡是同一件事，也跳过并在汇报里点名说明（不要默默丢，可能只是换了种说法）
- **带发布日期的灵感 = 直接排期**：如果某条灵感行里能识别出明确的发布日期（形如 `📅 2026-09-20`、`2026-09-20`、`9月20日`、`09/20` 等），则**跳过「选题池」的手动决策，直接排期**：
  1. 选题卡状态写 `🟢 已排期`，发布日期记进卡片「后续动作」
  2. 立即按 `时间管理/_早间排期指令.md` 的倒排期规则，倒推 写脚本/拍摄·素材整理/剪辑/发布 各环节日期（判断拍摄还是素材整理、剪辑几天，按 `_早间排期指令.md` 第一节第 4 点约定，卡上未标注则按内容判断，默认拍摄）
  3. 生成全部排期产物：3 份环节笔记（`环节档案/`）+ 4 个日历事件（`内容日历/流程日历/`）+ 4 条每日小记任务（`每日小记/`）+ 看板 4 张分卡
  4. 选题卡（原卡）直接勾选 `[x]` 放入看板「📦 已排期」列（不放进「📥 选题池」）
  5. 原灵感行仍按下方「处理完后」规则标记已处理
- **入库看板**（无日期的卡，生成选题卡后立即做）：每张新卡**自动推入看板「📥 选题池」列**，格式固定：
  - `- [ ] [[{选题卡文件名}]] · 📅 ____`
  - 在 `时间管理/3-内容流程看板.md` 的 `## 📥 选题池` 段末尾追加一行（标题以 `【示例·可删】` 开头的卡不推）
  - `📅 ____` 是**日期占位符**，用户决定排期时把 `____` 改成发布日期（如 `📅 2026-09-15`），agent 每天 9:05 识别这个日期自动倒排期；不写日期 = 还没排期打算，卡留在选题池
  - 已有同名行（选题卡名相同）→ 跳过不重复推
- **处理完后**（关键逻辑）：**不要删除原行、不要移动位置**：
  1. 把原行的 `- [ ]` 改为 `- [x]`（vault 需配 CSS 去掉勾选后的删除线，见部署 Step 3）
  2. 在原行**正下方**插入一行注释 `> ✅ 已处理 YYYY-MM-DD HH:MM → [[选题卡文件名]]`
  3. 保留原行在原地（用户能看到"这条录了什么 + 什么时候被处理 + 变成哪个卡"）
  4. 下次跑任务时 `- [x]` 行自动跳过，不会重复处理
- **汇报**：本次新增了几张卡、分别是什么、各带了哪段原始口述；推入看板「📥 选题池」几行；跳过了哪些（链接判重/口述判重/看板已有同名行）

```
## 三、验证

- 处理完后检查 `选题灵感库/` 是否有新卡
- 双击任意一张新卡：应能看到「🎙️ 原始口述」区，里面是当初在 `0-随手录入.md` 里口述/粘贴的原文，逐字保留
- `0-随手录入.md` 里处理过的行已打勾，下方带 `> ✅ 已处理` 注释
```

文件 3 · `时间管理/_早间排期指令.md`

```
# 自动排期（功能二 · Obsidian 内实现）

> 目标：用户在看板里决定"做哪条 + 哪天发"，9:05 自动任务读取信号 → 按环节耗时**倒排** 写脚本/拍摄·素材整理/剪辑/发布 → 写进 Obsidian（每日小记 + 内容日历）。排期结果由独立脚本 `wb-sync-ical.py`（每天 9:10 跑）单向同步到苹果系统日历，事件备注附 obsidian:// 链接。本指令**不在排期环节直接写苹果日历**。
> 状态流转：💡 灵感 → 🟡 可评估 → 🟠 已立项 → 🟢 已排期 → 🎬 脚本草稿 → 📹 拍摄中 → ✂️ 剪辑中 → ✅ 已发布

## 一、用户怎么操作（Obsidian 看板）

1. 打开 `时间管理/3-内容流程看板.md`（Kanban 视图）。
2. 在「📥 选题池」里决定要做哪条，**直接在卡上写发布日期**：卡片编辑状态下加 `📅 2026-09-10`（格式 YYYY-MM-DD）。没写日期 → 该卡停留在选题池，不会被自动排。
3. **写日期 = 决定排期**，9:05 自动任务读到带日期的卡就倒排期，排完原卡归档到「📦 已排期」。
4. **告诉 agent 这条用什么形式做**（可写在卡上，不写就按默认判断）：
   - 真人出镜/实拍 → 环节用「拍摄」；
   - 纯拆解/口播不露脸/录屏/网络素材混剪 → 环节用「素材整理」（**拍摄和素材整理二选一，永远不会同时出现**）；
   - 成片复杂、剪辑要 2 天 → 卡上写 `剪辑:2天`，不写默认 1 天。
5. 之后**不需手动拖动**——每天做完对应环节，去 `每日小记/YYYY-MM-DD.md` 勾掉对应 `#task`（任务行尾带 `→ [[环节档案/...]]` 跳转链接）。21:00 自动任务扫勾选状态，双写同步：①把 `内容日历/流程日历/` 对应事件笔记的 `complete: false → true`；②把看板对应卡片移到「✅ 已完成」列。

## 二、每天 9:05 自动排期任务指令

读取 `时间管理/3-内容流程看板.md`，找出「📥 选题池」列里**写了发布日期**的所有卡片，逐张处理：

0. **跳过示例**：标题以 `【示例·可删】` 开头的卡片一律忽略，不排期、不移动。
1. **读发布日期**：卡片上形如 `2026-09-10` 或 `📅 2026-09-10` 的日期即为发布日 P。没有日期 → 留在选题池，跳过不处理。
2. **判断环节序列**：
```

- 第二个环节是「拍摄」还是「素材整理」，看第一节第 4 点的约定（卡片标注优先，否则按内容判断，默认拍摄）；
- 剪辑天数：卡片写 `剪辑:2天` → 2 天；成片明显复杂（长成片/多素材/重特效）→ 2 天；其余默认 1 天。

3. **按耗时倒排**（每个环节单独占一整天，环节之间前后衔接不重叠，每天最多一个环节；最早环节在 `今天+1` 之前就开始的 → 不写，加入汇报"冲突"清单）：
   - **剪辑 1 天**：写脚本 P-3 → 拍摄/素材整理 P-2 → 剪辑 P-1 → 发布 P
   - **剪辑 2 天**：写脚本 P-4 → 拍摄/素材整理 P-3 → 剪辑 P-2、P-1 → 发布 P
4. **写入每日小记**：把各环节分别写成任务，追加到对应日期 `每日小记/YYYY-MM-DD.md` 末尾（文件不存在就创建，按 `模板/每日小记.md` 的骨架生成），每行格式：

   `- [ ] #task 内容·{卡片标题}-{写脚本/拍摄/素材整理/剪辑/发布} ⏳ YYYY-MM-DD HH:MM → [[环节档案/{环节笔记文件名}]]`
   - 剪辑占 2 天时，两天各写一条，日期不同；
   - 若某天已有任务，把时间错开、不重叠；
   - 写脚本那天的任务行尾可标预计时长，如 `（预计 3-5h）`，仅提示作用；
   - 发布环节任务行末尾**不挂链接**（发布不需要单独笔记）。

4.5 **生成环节笔记文件**：为每个非发布环节新建一份独立笔记到 `环节档案/` 目录：
   - **文件名**：`YYYY-MM-DD-{选题关键词(短)}-{环节}.md`（用环节对应日期 + 选题关键词 + 环节名），例：`2026-09-07-obsidian时间管理-写脚本.md`；
   - **格式（精简版，只生成 3 块）**：
     - **frontmatter**：`日期`（环节对应日期） / `星期` / `环节`（写脚本/拍摄/素材整理/剪辑） / `选题卡`（填 `[[选题灵感库/{原选题卡文件名}]]`） / `标签`：[内容生产] / `预计用时`（写脚本 3-5h；拍摄/素材整理 2-8h；剪辑 1-2 天 8h/天）
     - **正文**：单行 `> 💡 **一句话灵感:** {选题卡「一句话灵感」段原文}` + 空行 + `⏱ 预计用时 ...` + `## 🎬 镜头分镜表`（4 列：画面展示/口播文案/镜头时长/镜头描述，预留 5 行空表给用户填；表格下方必须带提示行：`> 💡 「画面展示」格可直接粘贴图片（自动存到 附件/），控制大小在文件名后加 \\|100`） + `## 🔗 关联`（上游 1 条 + 下游 1 条 + 选题 1 条） + `## ✅ 完成确认`（写脚本 3 条：大纲定稿/逐字稿完成/自读一遍无口误；拍摄 4 条：主镜头/备用镜头/素材导入/关键片段标记；剪辑 5 条：时间线/字幕/音频/封面/标题）
     - **不要替用户写创意内容**：分镜表等段落留空，用户当天打开笔记手填。`一句话灵感` 从选题卡复制，不另写。
   - **附件规则**：vault 已设 `attachmentFolderPath: "附件"`，用户在分镜表粘贴的图片自动存到 vault 根目录 `附件/`，不要改动该配置。

4.6 **挂链到选题卡**：打开原选题卡 `选题灵感库/{原选题卡文件名}.md`，在「后续动作」小节**下方**追加 `## 🗂 环节档案` 小节，列出全部环节笔记链接。已有同名小节则**整段覆盖刷新**，不叠加。

4.7 **生成日历事件笔记**：每个环节（含发布）在 `内容日历/流程日历/` 目录**新建一个事件笔记**，文件名 `YYYY-MM-DD {选题关键词}-{环节}.md`，frontmatter 固定格式：

```
---
title: {emoji}{环节} · {选题关键词}
date: YYYY-MM-DD
startTime: HH:MM
endTime: HH:MM
complete: false
---
```

→ [[环节档案/{环节笔记文件名}]]
```

- emoji：写脚本 📝 / 拍摄 🎬 / 素材整理 🎬 / 剪辑 ✂️ / 发布 🚀；发布环节正文只有 frontmatter，无链接行；
- endTime 按默认时长写：写脚本 10:00→15:00，拍摄/素材整理 09:30→16:30，剪辑 09:00→17:00，发布 18:00→18:30；
- 同名文件已存在 → 跳过不重复建。

5. **本环节不直接写苹果日历**：排期只落在 Obsidian。苹果日历由 `wb-sync-ical.py`（每天 9:10）单向同步，不要重复建苹果日历事件。
6. **生成看板分卡**（关键）：每个排好的选题在看板里生成 4 张独立的 `- [ ]` 行，分别落在 4 个流程列，每张一行、单行不换行（标题以 `【示例·可删】` 开头的卡不生成）：
   - **📝 写脚本 列**：`- [ ] 写脚本 · {选题关键词} · {MM-DD}({星期}) → [[环节档案/{YYYY-MM-DD}-{关键词}-写脚本]]`
   - **🎬 拍摄 列**：拍摄型写「拍摄」，素材整理型写「素材整理」（素材整理型也统一走 🎬 拍摄列，不单设列）
   - **✂️ 剪辑 列**：剪辑 1 天 → 1 行；剪辑 2 天 → 2 行（每天 1 行，日期不同）
   - **🚀 待发布 列**：`- [ ] 发布 · {关键词} · {MM-DD}({星期}) {HH:MM}`（默认 18:00；卡上指定时间则用卡上时间）
   - 每行必须保持单个 `- [ ]` 列表项、无换行；同列已有同名行（关键词相同）→ 跳过不重复追加
   - 选题关键词 = 选题卡文件名主体去掉日期前缀，例：`2026-09-04 古诗旧物.md` → 关键词 = `古诗旧物`

6.5 **排期完成后归档原卡**：该选题的 4 张分卡已落到流程列后，把「📥 选题池」列里那张原卡（含 `📅` 日期的行）**勾选 `[x]` 并移到「📦 已排期」列**（不删除，保留排期记录）。判断依据：该关键词已在「📝 写脚本」列出现同名分卡行 → 说明已排过期，跳过。

7. **每晚同步完成状态**（21:00 任务，双写）：扫 `每日小记/` 里所有「内容·」任务行（形如 `- [x] #task 内容·xxx-写脚本 ...`），找出已勾选 `[x]` 的，每条对应一处改动：
   - **改日历事件 frontmatter**：找到 `内容日历/流程日历/{YYYY-MM-DD xxx-环节}.md`，把 `complete: false` 改成 `complete: true`
   - **改看板 markdown**：找到 `时间管理/3-内容流程看板.md` 对应卡片行（关键词 + 环节名匹配），从所在列删掉；如果是「发布」环节 → 直接删；如果是其它环节 → 移到「✅ 已完成」列
   - 已勾选的卡片行不处理（已经是完成的）

8. **汇报**：为哪几张选题排了期（标题 + 发布日 + 用拍摄还是素材整理 + 各环节日期）；新建了哪些环节笔记；新建了哪些看板分卡；新建/更新了哪些日历事件；缺发布日期 / 排期冲突 / 过发布日未发布的选题分别列出来。

## 三、环节耗时与默认节奏（想改直接改这里，agent 每次重新读本文件）

| 环节 | 占用 | 每日工作时长 | 默认开始时间 | 备注 |
|---|---|---|---|---|
| 写脚本 | 1 天 | 3-5h | 10:00 | 无 |
| 拍摄 | 1 天 | 2-8h | 09:30 | 与素材整理二选一 |
| 素材整理 | 1 天 | 2-8h | 09:30 | 与拍摄二选一 |
| 剪辑 | 1-2 天 | 8h/天 | 09:00 | 默认 1 天；复杂成片或卡片写 `剪辑:2天` → 2 天 |
| 发布 | 0 天（发 P 当天） | 即时 | 18:00 | 所有流程结束后的当天发布 |
```

- 想留缓冲：在卡上把发布日期提前，或在卡上写 `缓冲+N`，agent 把最早环节整体再提前 N 天。
- 判断素材整理型的常见信号：视频不需要真人出镜（拆解、教程录屏、混剪、图文转视频）。判不了默认「拍摄」。

四、验证

- 打开当天及未来几天的 `每日小记/`：应看到 `#task` 任务行；
- 看板里该卡已归档，「📝 写脚本」列出现分卡；
- 点开 Full Calendar：周视图里应看到每个环节的事件块；
- 打开苹果系统日历 App：「{{CAL_NAME}}」里应出现同名事件，点开事件备注里带 obsidian:// 链接（由 `wb-sync-ical.py` 每天 9:10 同步）。

文件 4 · `时间管理/3-内容流程看板.md`

```
---
kanban-plugin: board
---

## 📥 选题池

- [ ] 【示例·可删】在这里写下你的第一个选题
- [ ] 【示例·可删】在卡片上写 `📅 2026-09-20` = 决定排期，次日 9:05 自动倒排


## 📦 已排期


## 📝 写脚本


## 🎬 拍摄


## ✂️ 剪辑


## 🚀 待发布


## ✅ 已完成
```

````
%% kanban:settings
```
{"kanban-plugin":"board","list-collapse":[]}
```
%%
````

（注意：看板卡片必须是单行 `- [ ]` 列表项，否则 Kanban 插件渲染出错。**用脚本写入看板后，已在 Obsidian 打开的用户需 Cmd+Q 重启 Obsidian**，Kanban 插件才会重新识别。）

文件 5 · `模板/选题灵感卡.md`

```
---
日期: {{date}}
星期: {{weekday}}
标签: [选题灵感, 口播]
状态: 💡 灵感
分类: 重磅模型/产品发布 | 行业大事件/突发热点 | 大佬/名人动态 | 深度教程/资源整理 | 行业观点/解读
来源: {{URL或情境}}
评分: 0
---

# {{title}}

## 一句话灵感
{{想到这个选题的瞬间，一句话说清是什么选题、为什么值得做}}

## 🎙️ 原始口述（原文逐字，不改写）
> 你当时说的话 / 贴的链接原文，逐字保留。双击本卡想看的"当初的原始素材"就在这里。
{{把 0-随手录入.md 里该条对应的原文整段贴进来；链接条目就贴那行的链接+备注。}}

## 想到的钩子
- 开头钩子：{{ }}
- 反转点：{{ }}
- 数字冲击：{{ }}

## 暂定切入角度
1. {{角度1}}
2. {{角度2}}
3. {{角度3}}

## 5 维评分（每维 ×5，总和 ×4 = 100）
```

```
| 时效 | 爆款 | 稀缺 | 成本 | 匹配 | 总分 |
|---|---|---|---|---|---|
| /5 | /5 | /5 | /5 | /5 | /100 |

等级：🔴S≥90 今日必写 | 🟠A 75–89 本周推荐 | 🟡B 60–74 储备 | ⚪C<60 暂缓

## 后续动作
- [ ] 排到本周计划

## 📂 环节档案
（排期后自动生成）

---
*状态流转：💡灵感 → 🟡可评估 → 🟠已立项 → 🟢已排期 → 🎬脚本草稿 → 📽️拍摄中 → ✂️剪辑中 → ✅已发布*
```

文件 6 · `模板/每日小记.md`

```
---
日期: {{date:YYYY-MM-DD}}
标签: [每日小记]
---

# {{date:YYYY-MM-DD}} 每日小记

## ✅ 待办
- [ ] {{ }}

## 📅 今日任务清单（自动）
> 时间管理系统里排到今天的任务会自动出现在这里（写法：`- [ ] #task 任务名 ⏳ 今天日期`），不用手动改。

```tasks
not done
scheduled today
sort by scheduled
```

💭 今日感悟
{{ }}
```

文件 7 · `.obsidian/snippets/no-task-strikethrough.css`

（作用：任务勾选后去掉删除线，只保留打勾视觉。没有它，「已处理」的灵感行会被划掉很难看。）

```css
body {
  --checklist-done-decoration: normal;
  --checklist-done-color: var(--text-muted);
  --checklist-done-opacity: 0.6;
}

.markdown-rendered .task-list-item.is-checked,
.markdown-rendered .task-list-item.is-checked p,
.markdown-rendered .task-list-item.is-checked * {
  text-decoration: none !important;
}

.HyperMD-task-line[data-task="x"],
.HyperMD-task-line[data-task="x"] * {
  text-decoration: none !important;
}

.task-list-item.is-checked s,
.task-list-item.is-checked del,
.HyperMD-task-line[data-task="x"] s,
.HyperMD-task-line[data-task="x"] del {
  text-decoration: none !important;
}

.task-list-item.is-checked {
  color: var(--text-muted) !important;
  opacity: 0.6;
}
.HyperMD-task-line[data-task="x"] {
  color: var(--text-muted) !important;
  opacity: 0.6;
}
```

Step 3 · 配置 Obsidian

1. **核心设置**：设置 → 文件与链接 → 附件默认存放路径 =「在指定的文件夹」→ 填 `附件`
2. **启用 CSS**：设置 → 外观 → CSS 代码片段 → 刷新 → 打开 `no-task-strikethrough` 开关
3. **安装并启用 6 个社区插件**（设置 → 第三方插件 → 关闭安全模式 → 浏览）：

插件	用途	关键配置
Kanban	内容流程看板	无需额外配置，看板文件已带 `kanban-plugin: board`
Tasks	任务勾选/查询	全局任务过滤器设为 `#task`

插件	用途	关键配置
Dataview	任务总览/时间线视图	开启 JavaScript 查询（后续按需）
Day Planner	每日小记的日历时间轴	模式选「Day Planner」
Full Calendar	周日历视图	添加「Full note」日历源，目录指向 `内容日历/流程日历`
Buttons	手动触发按钮（可…	高级用法，可先不装

4. **手机端**：iPhone 安装 Obsidian → 登录同一 Apple ID → vault 自动出现 → 把 `选题灵感库/0-随手录入.md` 设为星标

Step 4 · 部署自动化脚本与定时任务

把下面 4 个脚本放到 `~/.local/bin/`，全部 `chmod +x`。**先把占位符替换成第 3 节问到的答案。**

脚本 1 · `~/.local/bin/wb-import.sh`（9:00 灵感入库）

```
#!/bin/zsh
# 灵感入库：每天 9:00 由 launchd 触发
set -e

VAULT="{{VAULT_PATH}}"
AGENT="{{AGENT_CLI}}"
INSTRUCTION="$VAULT/选题灵感库/_早间入库说明.md"
LOG="/tmp/wb-import.log"

# launchd 环境没有用户 PATH，显式补上
export PATH="/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:$PATH"

echo "[$(date '+%F %T')] wb-import start" >> "$LOG"

"$AGENT" \
  -p "请严格按 $INSTRUCTION 第二节的流程跑灵感入库。读完后从 $VAULT/选题灵感库/0-随手录入.md 扫所有 - [ ] 未勾选行,生成选题卡;无发布日期的卡推入选题池,带发布日期的卡按「直接排期」规则:同时读 $VAULT/时间管理/_早间排期指令.md 执行倒排期(生成环节笔记/日历事件/每日小记任务/看板分卡),原卡勾选归档到「📦 已排期」列;最后标记原行已处理。汇报:新增了几张卡、哪些直接排期了、跳过了哪些。" \
  --add-dir "$VAULT" \
  >> "$LOG" 2>&1

echo "[$(date '+%F %T')] wb-import done" >> "$LOG"
```

脚本 2 · `~/.local/bin/wb-schedule.sh`（9:05 倒排期）

```
#!/bin/zsh
# 倒排期：每天 9:05 由 launchd 触发
set -e

VAULT="{{VAULT_PATH}}"
AGENT="{{AGENT_CLI}}"
INSTRUCTION="$VAULT/时间管理/_早间排期指令.md"
LOG="/tmp/wb-schedule.log"

export PATH="/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:$PATH"

echo "[$(date '+%F %T')] wb-schedule start" >> "$LOG"

"$AGENT" \
  -p "请严格按 $INSTRUCTION 跑早间排期任务。扫「📫 选题池」列所有写了 📅 日期的卡,按节奏倒推各环节日期,生成日历事件到 $VAULT/内容日历/流程日历/、环节笔记到 $VAULT/环节档案/、每日小记 #task,看板追加 4 张分卡,原卡勾选归档到「📦 已排期」。没写日期的卡留在选题池不动。汇报:为哪几张选题排了期、缺日期/冲突的选题。" \
  --add-dir "$VAULT" \
  >> "$LOG" 2>&1

echo "[$(date '+%F %T')] wb-schedule done" >> "$LOG"
```

脚本 3 · `~/.local/bin/wb-status.sh`（21:00 状态同步）

```
#!/bin/zsh
# 状态同步：每天 21:00 由 launchd 触发，扫每日小记已勾选任务 → 同步看板和日历
set -e

VAULT="{{VAULT_PATH}}"
AGENT="{{AGENT_CLI}}"
LOG="/tmp/wb-status.log"

export PATH="/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:$PATH"

echo "[$(date '+%F %T')] wb-status start" >> "$LOG"

"$AGENT" \
  -p "请严格按 $VAULT/时间管理/_早间排期指令.md 第 7 节「每晚同步完成状态」执行:扫 $VAULT/每日小记/ 里已勾选 [x] 的 #task 任务行,把对应 $VAULT/内容日历/流程日历/ 事件笔记的 complete 改为 true,并把 $VAULT/时间管理/3-内容流程看板.md 对应卡片移到「✅ 已完成」列(发布环节直接删除该行)。汇报处理了哪些任务。" \
  --add-dir "$VAULT" \
```

```
  >> "$LOG" 2>&1

echo "[$(date '+%F %T')] wb-status done" >> "$LOG"
```

脚本 4 · `~/.local/bin/wb-sync-ical.py`（9:10 同步苹果日历）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Obsidian 内容排期 → 苹果系统日历单向同步
- 扫 内容日历/流程日历/*.md 的 frontmatter
- 先删日历里 description 含标记 [Obsidian内容] 的旧事件，再重建
- 每个事件 description 附 obsidian:// 链接
由 launchd 每天 9:10 触发，也可手动跑
"""
import os, re, subprocess, urllib.parse, tempfile

VAULT = "{{VAULT_PATH}}"          # vault 绝对路径
CAL_DIR = os.path.join(VAULT, "内容日历/流程日历")
TOPIC_DIR = os.path.join(VAULT, "选题灵感库")
VAULT_NAME = "{{VAULT_NAME}}"     # vault 显示名（obsidian:// 链接用）
CAL_NAME = "{{CAL_NAME}}"         # iCloud 下的目标日历名
MARKER = "[Obsidian内容]"
LOG = "/tmp/wb-sync-ical.log"

def log(msg):
    import datetime
    with open(LOG, "a", encoding="utf-8") as f:
        f.write(f"[{datetime.datetime.now():%F %T}] {msg}\n")

def obsidian_url(note_path):
    return "obsidian://open?vault=" + urllib.parse.quote(VAULT_NAME) +
"&file=" + urllib.parse.quote(note_path)

def parse_frontmatter(content):
    fm = {}
    m = re.match(r'^---\n(.*?)\n---', content, re.DOTALL)
    if m:
        for line in m.group(1).splitlines():
            if ':' in line:
                k, v = line.split(':', 1)
                fm[k.strip()] = v.strip()
    return fm

def topic_keyword(title):
```

```
    if '·' in title:
        return title.split('·', 1)[1].strip()
    return title.strip()

def find_topic_note(keyword):
    if not os.path.isdir(TOPIC_DIR):
        return None
    for fname in sorted(os.listdir(TOPIC_DIR)):
        if fname.endswith('.md') and keyword in fname:
            return fname[:-3]
    return None

def collect_events():
    events = []
    for fname in sorted(os.listdir(CAL_DIR)):
        if not fname.endswith('.md'):
            continue
        path = os.path.join(CAL_DIR, fname)
        with open(path, encoding='utf-8') as f:
            content = f.read()
        fm = parse_frontmatter(content)
        title = fm.get('title', fname[:-3])
        date = fm.get('date', '')
        start = fm.get('startTime', '')
        end = fm.get('endTime', '')
        complete = (fm.get('complete', 'false') == 'true')
        links = re.findall(r'\[\[([^\]]+)\]\]', content)
        link = links[0].split('|')[0] if links else None
        if not link:
            topic = find_topic_note(topic_keyword(title))
            link = f"选题灵感库/{topic}" if topic else None
        if not date or not start:
            continue
        events.append({'title': title, 'date': date, 'start': start,
                       'end': end, 'complete': complete, 'link': link})
    return events

def build_jxa(events):
    lines = []
    lines.append('const app = Application("Calendar");')
    lines.append(f'const cal = app.calendars.byName("{CAL_NAME}");')
    lines.append('const evs = cal.events();')
    lines.append('for (const e of evs) {')
    lines.append('  try { if ((e.description()||"").includes("' + MARKER +
'")) app.delete(e); } catch(err){}')
    lines.append('}')
```

```
    for ev in events:
        summary = ("✓ " if ev['complete'] else "") + ev['title']
        def esc(s):
            return s.replace('\\', '\\\\').replace('"', '\\"').replace('\n',
'\\n')
        desc = MARKER + "\n" + (obsidian_url(ev['link']) if ev['link'] else
"")
        start_iso = f"{ev['date']}T{ev['start']}:00"
        end_iso = f"{ev['date']}T{ev['end']}:00"
        lines.append(f'cal.events.push(app.Event({{summary:"
{esc(summary)}",startDate:new Date("{start_iso}"),endDate:new Date("
{end_iso}"),description:"{esc(desc)}"}}));')
    return '\n'.join(lines)

def main():
    events = collect_events()
    log(f"解析到 {len(events)} 个排期事件")
    jxa = build_jxa(events)
    tmp = tempfile.NamedTemporaryFile(mode='w', suffix='.js', delete=False,
encoding='utf-8')
    tmp.write(jxa)
    tmp.close()
    try:
        r = subprocess.run(['osascript', '-l', 'JavaScript', tmp.name],
                           capture_output=True, text=True, timeout=300)
        if r.returncode != 0:
            log(f"osascript 错误: {r.stderr.strip()}")
            print(f"同步失败: {r.stderr.strip()}")
            return 1
        log(f"同步成功,共 {len(events)} 个事件")
        print(f"同步成功: {len(events)} 个事件写入「{CAL_NAME}」")
        return 0
    finally:
        os.unlink(tmp.name)

if __name__ == '__main__':
    raise SystemExit(main())
```

（首次运行会弹出「日历」权限申请，必须点允许；如果目标日历名不存在会报错——先确认 `{{CAL_NAME}}` 已在苹果日历 App 的 iCloud 账户下建好。）

注册 launchd 定时任务

在 `~/Library/LaunchAgents/` 下创建 4 个 plist（Label 分别为 `com.content.import` / `com.content.schedule` / `com.content.sync-ical` / `com.content.status`）。模板如下，替换 Label / 脚本路径 / 触发时间：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.content.import</string>

    <key>ProgramArguments</key>
    <array>
        <string>/bin/zsh</string>
        <string>/Users/你的用户名/.local/bin/wb-import.sh</string>
    </array>

    <!-- 触发时间 -->
    <key>StartCalendarInterval</key>
    <dict>
        <key>Hour</key>
        <integer>9</integer>
        <key>Minute</key>
        <integer>0</integer>
    </dict>

    <key>StandardOutPath</key>
    <string>/tmp/wb-import-launchd.log</string>
    <key>StandardErrorPath</key>
    <string>/tmp/wb-import-launchd.log</string>

    <key>RunAtLoad</key>
    <false/>
</dict>
</plist>
```

四个任务的触发时间表：

Label	脚本	时间
com.content.import	wb-import.sh	9:00
com.content.schedule	wb-schedule.sh	9:05
com.content.sync-ical	wb-sync-ical.py	9:10
com.content.status	wb-status.sh	21:00

注册并启动：

```
launchctl load ~/Library/LaunchAgents/com.content.import.plist
launchctl load ~/Library/LaunchAgents/com.content.schedule.plist
launchctl load ~/Library/LaunchAgents/com.content.sync-ical.plist
launchctl load ~/Library/LaunchAgents/com.content.status.plist
```

Step 5 · 试运行验证（部署完成前必做）

1. 在手机或电脑的 `0-随手录入.md` 里写一条测试灵感：`- [ ] 测试选题，9月30日发布`
2. 手动跑 `~/.local/bin/wb-import.sh`，等待完成后检查 **5 处产物**：
   - `选题灵感库/` 出现 `YYYY-MM-DD 测试选题.md`（状态 🟢 已排期，含原始口述）
   - `环节档案/` 出现 3 份环节笔记（写脚本/拍摄或素材整理/剪辑）
   - `内容日历/流程日历/` 出现 4 个事件笔记
   - `每日小记/` 对应日期出现 `#task` 任务行
   - 看板「📦 已排期」出现归档卡，「📝 写脚本」等 4 列出现分卡
3. 手动跑 `~/.local/bin/wb-sync-ical.py`，检查苹果日历 App「{{CAL_NAME}}」出现 4 个事件，点开备注有 obsidian:// 链接
4. 在每日小记里勾掉一个测试任务，手动跑 `~/.local/bin/wb-status.sh`，检查看板卡片移入「✅ 已完成」
5. 全部通过 → 删除测试数据（测试卡、环节笔记、日历事件、看板行），告诉用户：**明天早上 9 点开始，系统全自动运行**

5. 验收清单（全绿 = 部署成功）

- [ ] vault 在 iCloud 路径下，手机能看到并已星标「0-随手录入」
- [ ] 6 个插件装好，看板正常渲染，CSS 去删除线生效
- [ ] 4 个脚本可手动跑通，产物齐全
- [ ] 4 个 launchd 任务已 load（`launchctl list | grep com.content` 可见）
- [ ] 苹果日历事件带 obsidian:// 链接，手机点开能跳回 Obsidian
- [ ] 试运行 5 处产物验证通过，测试数据已清理

附录 · 常见问题

Q：Mac 关机/睡眠定时任务还跑吗?
不跑。launchd 只在开机时触发当天错过的日历任务一次（默认行为），建议睡前不关机或接受偶尔跳过——灵感不会丢，下次开机 9:00 会一并处理。

Q：手机上改了灵感，电脑多久能读到?
iCloud 通常几秒内同步；偶尔延迟，9:00 前同步到即可。

Q：看板卡片显示成乱码/纯文本？
卡片必须是单行列表项；改完看板文件后重启 Obsidian（Cmd+Q 再打开）。

Q：日历事件重复了？
`wb-sync-ical.py` 每次全量重建带 `[Obsidian内容]` 标记的事件，重复说明目标日历名配错或手动建过同名事件。检查 `CAL_NAME`。

Q：想改环节节奏/时长？
直接编辑 `时间管理/_早间排期指令.md` 第三节的表格，agent 每次任务都会重新读它——改文档就是改规则，不用碰代码。